

**INSITUT SUPERIEUR DE STATISTIQUE ET DES
NOUVELLES TECHNOLOGIES**

COURS DE GENIE LOGICIEL

Année académique : 2024-2025

CHAPITRE I : INTRODUCTION SUR LE GENIE LOGICIEL

I.1 OBJECTIF DU COURS

I.2.1 Objectif général

Le présent cours a pour objectif général : - Décrire et appliquer les différentes méthodes et techniques de développement d'un logiciel de qualité.

I.2.2 Objectif spécifiques

- Définir ce qu'est le Génie Logiciel, en comprendre les enjeux ;
- Expliquer le cycle de vie d'un logiciel ;
- Utiliser les méthodes et techniques de développement logiciel en se focalisant sur les approches Orienté Objets et composants ;
- Comparer les différentes méthodes de développement logiciel (Classiques et agiles) ;
- Evaluer les besoins du client ;
- Elaborer un modèle d'analyse détaillé à partir d'un système existant ;
- Analyser un Système d'Information existant et en proposer un modèle adéquat au besoin du client ;
- Appliquer quelques Design Pattern (Patron de Conception) dans le développement logiciel ;
- Évaluer et maintenir un logiciel.

I.2 DEFINITION

I.2.1 Génie Logiciel Retenons que l'on peut donner plusieurs définitions du concept de Génie Logiciel (En anglais software engineering), et cela peut dépendre d'un auteur à un autre, d'un enseignant à un autre ou d'un développeur à un autre. C'est ainsi que nous allons donner un certain nombre des définitions des différents auteurs mais qui au final aboutissent au même. Le génie logiciel est un domaine des sciences de l'ingénieur dont l'objet d'étude est la conception, la fabrication, et la maintenance des systèmes informatiques complexes. Le génie logiciel est un ensemble de méthodes, techniques et outils pour la production et la maintenance de composants logiciels de qualité. Le génie logiciel est une science de génie industriel qui étudie

les méthodes de travail et les bonnes pratiques des ingénieurs qui développent des logiciels. Pour notre cours, nous allons nous contenter de cette définition très concise définissant le Génie Logiciel comme étant « L'art et la manière de créer un logiciel ». En parlant de logiciel, il en va des petits au moyens, jusqu'à arriver aux grands logiciels : Tout en ne faisant abstraction d'aucun domaine des sciences susceptible d'utiliser un quelconque logiciel ; il s'agit notamment de la Comptabilité, des Ventes en ligne ou non, de l'électronique automobile ou navale ou ferroviaire ou aérienne, de la Construction, de la Médecine, de la Circulation routière, des équipements de télécommunication ou informatique (Embarqués ou non), des Drones, etc ; bref des Nouvelles Technologies de l'Information et de la Communication. C'est ainsi que nous constaterons que le domaine du Génie Logiciel est très vaste et très varié suivant l'orientation de l'ingénieur : C'est la raison pour laquelle nous devons être certains de ne pas pouvoir aborder tous les domaines ayant trait à ce sous-domaine de la science, et par la même occasion, nous ne pourrions pas non plus explorer tout ce qui serait relatif à celui-ci.

I.2.2 Logiciel

Rappelons qu'un logiciel ou programme existe sous deux formes. D'abord, il s'agit d'un texte écrit par un être humain dans un langage de programmation : c'est ce que l'on appelle le code source. C'est ensuite, éventuellement, un code exécutable, compréhensible par des machines. Certains programmes sont lus et interprétés à la volée : on parlera des langages interprétés ; d'autres ont besoin d'être préparés pour être compris plus vite par une machine et intégrés à leur environnement : c'est la compilation, qui, à partir du code source, produit un code exécutable. En effet, la plupart des gens ne voient pas du code source. Acheter un logiciel consiste en fait à acheter un support physique sur lequel se trouve du code exécutable. Bref, un logiciel est un code source ou code exécutable ayant une fonction bien précise : logiciel de comptabilité, logiciel de gestion des prêts, etc. En outre, ce dernier peut être exécuté (Au cas de l'exécutable) sur un ordinateur, une tablette ou un téléphone ; ou être embarqué dans un système tout à fait autonome (Véhicule, Climatiseur, Montre, etc.). Dans un sens plus large ; c'est un programme ou ensemble de programmes permettant à un ordinateur ou à un système informatique d'assurer une tâche ou une fonction particulière. En parlant de logiciel, on peut en distinguer deux types :

1. Logiciel propriétaire : Est un logiciel dont le contrat de licence stipule que seul l'auteur du logiciel est agréé de le modifier. Avec cette catégorie de logiciel, vous ne pouvez en aucun cas user du code source du dit logiciel, vous ne pouvez ni l'améliorer, ni le redistribuer ou même en

faire commerce (Revendre). Ce genre de logiciel fait intervenir ce que l'on appelle le copyright (©) ou droit d'auteur.

Exemple : Tous les logiciels de la firme Microsoft (MS Office, MS SQL Server, Windows Server quelque soit sa version, etc.), tous les logiciels de la firme Apple (iTunes pour iOS, iPhone et iMac pour la gestion des musiques et autres ; iBooks ; Siri utilitaire intelligent, FaceTime pour appel vidéo, etc.), les logiciels de la collection Adobe (Adobe photoshop, Adobe Illustrator, Adobe After Effect, etc.), SAP ERP (Logiciel Allemand de gestion intégré), etc.

2. Logiciel libre : Est un logiciel dont l'auteur a rendu les secrets de fabrication (Code source) librement accessible et librement utilisable. Ici, le contrat de licence stipule qu'on peut modifier le code source au besoin. Ce genre de logiciel par contre, fait intervenir ce que l'on appelle le copyleft (Symbole inversé du Copyright) ou droit d'auteur inversé.

Exemple : Le projet GNU/GPL sur lequel est fondé bon nombre des distributions Linux ainsi que ses logiciels applicatifs : Ubuntu, BackTrack, CentOS, etc. Après avoir donné un aperçu sur un logiciel, on pourrait alors se demander que ce qu'un Progiciel ? Etant donné que c'est un terme assez courant en informatique. Un Progiciel n'est rien d'autre qu'un logiciel généraliste et multifonction. Un exemple courant serait le Bloc Office de Microsoft et même SAP ERP.

4 I.2.2.1 Catégories de logiciel

Partant des différents types de logiciels, nous pouvons en distinguer cinq (5) catégories qui sont les suivantes :

1. Logiciel sur mesure (Custom en anglais) : Est un logiciel personnalisé, c'est – à – dire fait pour un client bien spécifique et pour un besoin donné. Exemple : Le logiciel ISSNT, ou tout autre logiciel de gestion spécifique à une entreprise donnée.
2. Logiciel générique (Generic en anglais) : Est un logiciel vendu sur le marché, telle un logiciel de traitement de texte, un tableur, logiciel de base des données, etc. Exemple : MS Excel, MS Access, MS Word, etc.
3. Logiciel embarqué (Embedded en anglais) : Est un logiciel scellé dans un matériel électronique. Exemple : Montre, Robots, Machine à café, Téléviseur, etc.
4. Logiciel en temps réel (real – time en anglais) : Est un logiciel prenant en compte de façon la plus rigoureuse possible les contraintes de temps en des délais imposés. Elle assure le contrôle et la surveillance avec réaction immédiate requise. Généralement, il est soumis à

des environnements très contraignant. Ces genres de logiciel sont souvent embarqués. Exemple : Logiciel de navigation dans un avion, Contrôleur du pilote automatique (Que ce soit sur avion ou autre engin), Logiciel de gestion des réacteurs nucléaires, Logiciel de gestion des robots en industries, Système de guidage des missiles, etc.

5. Logiciel de traitement de données (Data processing en anglais) : Est un logiciel capable de traiter des grandes quantités des données tout en facilitant leurs accès sécurisés, pouvant même arriver à leur analyse et une projection sur ces données.

Exemple : Logiciels médicaux, Logiciels d'analyse des données (Météorologique, Sismique, Volcaniques, etc.), Logiciels statistique (SPSS, etc.), logiciel simulant le Data mining (Analyse prédictive et d'exploitation des grandes quantités de données le Big Data Analytic), etc. Suivant qu'un logiciel devient de plus en plus important, il prend de l'ampleur du point de vue de son développement, et cela va de pair avec la catégorie dans laquelle elle se trouve. C'est ainsi qu'un logiciel générique par exemple demanderait un plus grand effort de développement par rapport à un logiciel sur mesure, ou un logiciel en temps réel serait plus exigeant à produire qu'un logiciel embarqué par exemple.

De nos jours la plupart des logiciels que nous rencontrons ne sont plus conçu en architecture 1/3, mais tendent à des architecture multi – tiers, faisant intervenir un concept d'architecture distribuée.

I.2.2.2 Matériel et logiciel

Actuellement bon nombre des matériels (Ordinateur, Machine à laver, Jouets, etc.) que nous trouvons sur le marché sont standardisés et fiables, c'est – à – dire faits suivant des normes et des standards de fabrication ; ce qui fait que les constructeurs s'y conforment rapidement. Pourtant, presque tous ces matériels ne peuvent pas fonctionner sans des logiciels, qui sont un réel problème pour l'informatique : Il est difficile de faire pour le logiciel ce qui est fait pour le matériel. C'est la raison pour laquelle, le logiciel est pratiquement au cœur des Nouvelles Technologies de l'Information et de la Communication et est le seul élément qui peut bel et bien bloquer toute la machine de développement technologique ; car sans le logiciel, le matériel est presque inutilisable. Cet état de chose, nous pousse à plonger dans la quête du meilleur logiciel pouvant fonctionner avec un matériel quelconque que l'on dispose : ce qui conduit au pourquoi du Génie Logiciel.

I.3 POURQUOI LE GENIE LOGICIEL

I.3.1 Historique

A la fin des années 1960 éclate la « crise du logiciel », prise de conscience des difficultés que rencontre le développement des grands projets informatiques. De ce constat va naître une nouvelle discipline, le génie logiciel. La fin des années 1950 voit la création des premiers langages de programmation de haut niveau. Dans la période de jouissance qui suit, on pense que l'usage de ces nouveaux langages, qui viennent se substituer à l'assembleur, va résoudre les problèmes du développement des applications informatiques. Le terme de « programmation automatique », en vogue à l'époque, est représentatif de cet état d'esprit. Mais la réalité ne va pas tarder à se manifester : Celle de la crise du logiciel. Dans les années 1960, l'informatique conquiert de nouveaux champs d'application. Initialement centrée sur le calcul scientifique, elle s'impose dans la gestion des entreprises, puis dans la commande de procédés industriels (Chimie, Nucléaire, Production, etc.), ainsi que dans les applications embarquées (Aviation, Spatial, Communication, Domaine militaire, etc.). Avec la naissance des sociétés de services informatiques, la production de logiciel, initialement concentrée chez les constructeurs d'ordinateurs et les grands utilisateurs, devient l'objet d'une industrie spécifique. Au cours de cette période, la taille des projets informatiques connaît une croissance exponentielle. Ainsi, la taille (en milliers de lignes de code) du logiciel de base des machines IBM croît de 25 en 1958 (IBM 704) à 500 en 1964 (IBM 7090) puis à 5 000 en 1968 (IBM 360). Leur complexité, dont une estimation grossière est le nombre d'interactions entre les différents composants qui constituent un logiciel, croît en proportion. Vers la fin des années 1960, un constat s'observe : le développement des grands projets informatiques est de moins en moins bien maîtrisé. Les estimations de coût et de délai de production se révèlent inférieures à la réalité, souvent d'un facteur de 2 à 4. Les équipes de programmeurs sont confrontées à des problèmes de communication que l'encadrement peine à résoudre. Enfin, les logiciels produits ne répondent pas toujours aux attentes ; il faut dire que les spécifications ne capturent pas toujours les besoins de manière adéquate, faute d'outils et de méthodes adaptés. La tâche de maintenance, qui devrait normalement consister à adapter le logiciel à l'évolution des besoins, vise surtout à corriger ces fautes, sans toujours éviter d'en introduire des nouvelles. Face à ce sombre tableau, une prise de conscience se manifeste dans la communauté informatique : la production des logiciels manque du fondement rationnel présent dans d'autres techniques plus anciennes. De manière analogue au génie civil, au génie électrique et au génie chimique, il faut promouvoir un

génie logiciel. L'initiative viendra de la division des affaires scientifiques de l'OTAN (Organisation du Traité de l'Atlantique Nord), qui organise en octobre 1968 sur la suggestion de F.L. Bauer, professeur à l'université technique de Munich ; une conférence de travail sur les difficultés de la production du logiciel et les moyens de les surmonter. Intitulée Working Conference on Software Engineering, elle est considérée comme l'événement fondateur de cette nouvelle discipline et c'est elle qui popularise le terme de software engineering, traduit en français par « génie logiciel ». 7 Cette conférence (Software Engineering 68) a eu lieu à Garmisch, en Allemagne, et réunit plus de 50 participants (usagers de l'informatique, industriels et chercheurs). Elle sera suivie d'une seconde conférence (Software Engineering) sur le même thème, à Rome, en octobre 1969. Ces deux conférences n'ont pas apporté des solutions immédiates à la crise du logiciel, ce n'était pas leur but ; mais elles ont permis de faire le point sur la situation du domaine, d'identifier les grandes lignes de la discipline naissante du génie logiciel et de définir quelques-uns de ses objectifs ainsi que des pistes de recherche. Si la conférence SE a avant tout révélé l'étendue de la crise du logiciel, SE a quant à elle approfondi certains aspects techniques, et a surtout mis en évidence des difficultés de communication au sein de la communauté : entre théoriciens et praticiens, entre chercheurs et industriels, entre constructeurs et utilisateurs. Ces problèmes n'étaient que le reflet de ceux du monde extérieur mais ils n'étaient pas attendus et, de ce fait, leur analyse constitua un thème majeur des discussions.

1.3.2 Le génie du Logiciel

Un logiciel est tout d'abord conçu pour répondre à un besoin spécifique de l'utilisateur : Et cela tout en respectant les contraintes de temps et des ressources à sa disposition (Ressources Humaine, matériels et financière), sur lesquels reposent la satisfaction ou l'insatisfaction du client. C'est l'association de ces différents éléments qui a induit l'idée du génie : qui relève du respect d'un grand nombre d'éléments devant aboutir à un seul idéal, qui n'est rien d'autre que livrer au client ce qu'il a demandé ou résoudre le problème du client (Utilisateur). En outre, on parle de l'ingénierie lorsqu'on doit faire face à la résolution d'un problème pour lequel les ressources à utilisées sont limitées (Temps, Argent, etc.), on recherche une solution utile résolvant un problème concret, le problème n'est pas inventé par l'ingénieur, il y a rigueur dans la résolution du problème et la prédictibilité du résultat.

Une étude par le Standish Group en 1995 sur 8 380 projets, a révélé ceci :

- Succès : 16 % ;
- Problématique : 53 % (Budget ou délais non respectés, défaut de fonctionnalités) ;
- Echec : 31 % (Abandonnés).

C'est alors une raison suffisante de génie de pouvoir mener à bien un projet informatique qui serait non seulement accepté par le client à bras ouvert, mais aussi et surtout qui respecterait le délai imparti avec pratiquement les mêmes ressources au départ du projet ; que ce soit humaines, matérielles ou financières. Même si ces chiffres ont évolué, il reste toutefois évident que le pourcentage de réussite d'un projet informatique reste encore faible (A peu près 29 %). Aussi, il se remarque que le taux de succès décroît avec la taille des projets et la taille des entreprises.

I.4 LES QUATRE (4) P DU GENIE LOGICIEL

- Personnel : Qui produit le logiciel ? Ici, on détermine les différentes ressources humaines devant intervenir dans le projet, ainsi que l'équipe en sa responsabilité ;
- Processus : Comment le logiciel est-il produit ? Ici, on détermine les différentes étapes à suivre pour la conception du produit ;
- Projet : La production réelle du logiciel. Il s'agit, à ce niveau de la programmation proprement dites, c'est l'implémentation informatique de la solution attendue par le client ;
- Produit : A ce niveau, on enrichie le logiciel (Produit fini) par la documentation et toutes sorte de spécifications utiles au dit logiciel.

Tous les objets fabriqués pendant la production ; - Code source, exécutables, documentation, modèles de conception, cahier de charges, résultats de tests, mesure de productivité, etc.

I.5 PARTIES PRENANTES DANS LE GENIE LOGICIEL

Il y a généralement quatre groupes d'acteurs prenant part au développement d'un logiciel. Il s'agit de :

1. Utilisateurs (Users en Anglais) : Sont ceux qui se servent du logiciel. C'est le consommateur du produit final ou le bénéficiaire du logiciel.

2. Clients (Customers en anglais) : Sont ceux qui paient pour le logiciel. Il peut constituer un seul individu (Morale ou physique) ou un groupe d'individu. C'est ainsi qu'une entreprise peut par exemple acheter un logiciel que ses agents vont utiliser.

3. Développeurs (Developers en anglais) : Sont ceux qui conçoivent le logiciel. Ils tapent les différentes lignes de codes qui constitueront le produit fini après validation des tests. Leur nombre varie suivant l'ampleur du projet : Il peut aller d'un à plusieurs milliers des développeurs.

4. Gestionnaires (Managers en anglais) : Sont ceux qui supervisent la production du logiciel pour son bon aboutissement dans le délai et respectant les besoins de l'utilisateur final qui peut ne pas être le client.

I.6 PROCESSUS DE DEVELOPPEMENT D'UN LOGICIEL

Un processus caractérise l'ensemble d'activités coordonnées et régulées, en partie ordonnées, dont le but est de créer un produit. Le processus de développement est l'ordre et la manière d'enchaîner les étapes d'un développement. C'est ainsi que pour arriver à bon port dans un projet de Génie Logiciel, il est idéal de suivre une certaine démarche pouvant permettre un respect de délai d'exécution avec des coûts réduits. Un processus décrit 2 choses importantes :

1. Les activités (étapes) : Quoi faire ?

2. L'enchaînement des activités : Quand faire l'activité précise ? Un logiciel à développer est un problème très complexe à résoudre. Pour appréhender cette complexité, il faut arriver à rendre les choses simples. Pour cela, il faut séparer les problèmes qui peuvent l'être afin de les résoudre de manière presque indépendante : Analogie du principe « Diviser pour mieux régner ». Un processus définit un enchaînement d'activités pendant lesquelles on traite des problèmes différents. Ces problèmes ne sont pas complètement indépendants, c'est pourquoi l'enchaînement est important. Voici de façon détaillée l'enchaînement à suivre :

1. Etude de faisabilité : Déterminer si le développement proposé vaut la peine d'être mis en œuvre, compte tenu des attentes et de la difficulté de développement ;

2. Spécification du logiciel : Définit ses fonctionnalités et ses contraintes. Permet d'obtenir de l'utilisateur ses exigences pour le logiciel ;

3. Analyse : Comprendre les exigences en profondeur en construisant des modèles. Le but de l'analyse est de spécifier le quoi : Ce qui doit être réalisé et non le comment ou la façon de le réaliser. Il s'agit de comprendre le problème avant d'essayer de lui trouver une solution ;

4. Conception du logiciel : Déterminer la structure et l'architecture du système, ainsi que la façon dont les différentes parties vont communiquer entre – elles (Interfaces). Ajuster les modèles du monde réel obtenus par l'analyse pour qu'ils soient compatibles avec une implémentation informatique : C'est à ce stade que l'on se fixe le langage de programmation à utiliser ;

5. Implémentation : Traduire la conception en code et en structures de bases de données. En outre, pour s'assurer d'une bonne validation du logiciel par l'utilisateur, il est crucial de le valider graduellement au fur et à mesure de son implémentation ;

6. Tests et Validation : Vérifier que l'application est réellement utilisable et qu'elle correspond vraiment aux exigences de l'utilisateur. C'est toujours l'utilisateur qui valide l'application tout en vérifiant qu'il fonctionne suivant son besoin ;

7. Déploiement : Mettre l'application en production et remplacer élégamment les applications existantes ;

8. Formation : Permettre aux utilisateurs de maîtriser le nouveau logiciel mis en place ;

9. Maintenance : Préserver la pérennité de l'application. C'est l'étape la plus cruciale et la plus coûteuse d'un projet informatique. Elle est vue sur deux aspects (Maintenance curative : Maintenance corrective et adaptative, et Maintenance préventive : maintenance perfective ou additive) en trois volets :

- Maintenance corrective : Corriger des défauts (Bugs informatique) ;

- Maintenance adaptative : Modifications à apporter au système de façon à tenir compte de changement dans : Le Système d'Exploitation ; La plate-forme ; Les données ou la base de données ; Les règles et procédures, etc.

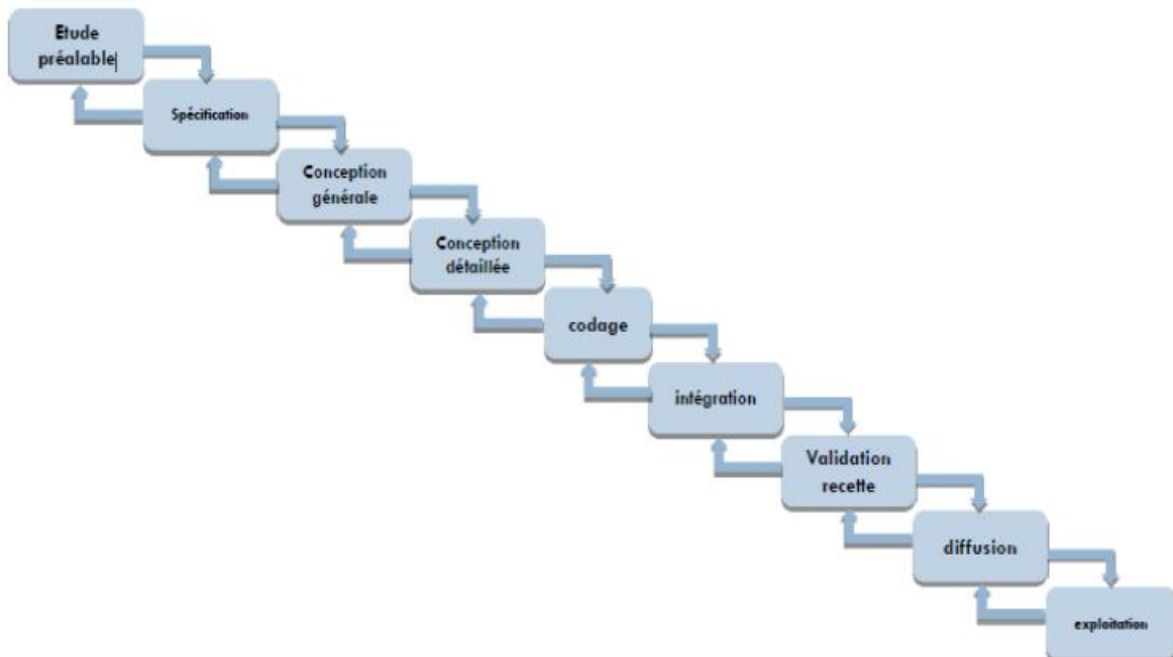
- Maintenance perfective ou additive : ajouter de nouvelles exigences ou des nouvelles fonctionnalités.

CHAPITRE II : MODELES DE CONCEPTION DES LOGICIELS

Pour que le processus de développement d'un logiciel aboutisse correctement, il est important de suivre un certain nombre des modèles qui caractérisent le cycle de vie du développement d'un logiciel.

II.1 MODELE EN CASCADE

L'approche en cascade exige que les différents stades du développement logiciel suivent une séquence linéaire et rigide qui n'admet pas des retours en arrière. Les développeurs commencent par recueillir les exigences, puis ils construisent un modèle d'analyse et réalisent la conception du système. Ils conçoivent ensuite les classes puis passent à l'implémentation, au développement, aux tests et au déploiement. Chaque stade est intégralement terminé avant que le suivant puisse débuter. L'approche en cascade convient aux applications bien comprises, quand les résultats de l'analyse et de la conception sont prévisibles, mais de telles applications sont rares.



1. Etude préalable (Feasibility) : C'est une phase exploratoire dans laquelle on fait un état de lieu de la faisabilité du logiciel. Déterminer si le développement proposé vaut la peine d'être mis en œuvre, compte tenu des attentes et de la difficulté de développement ;

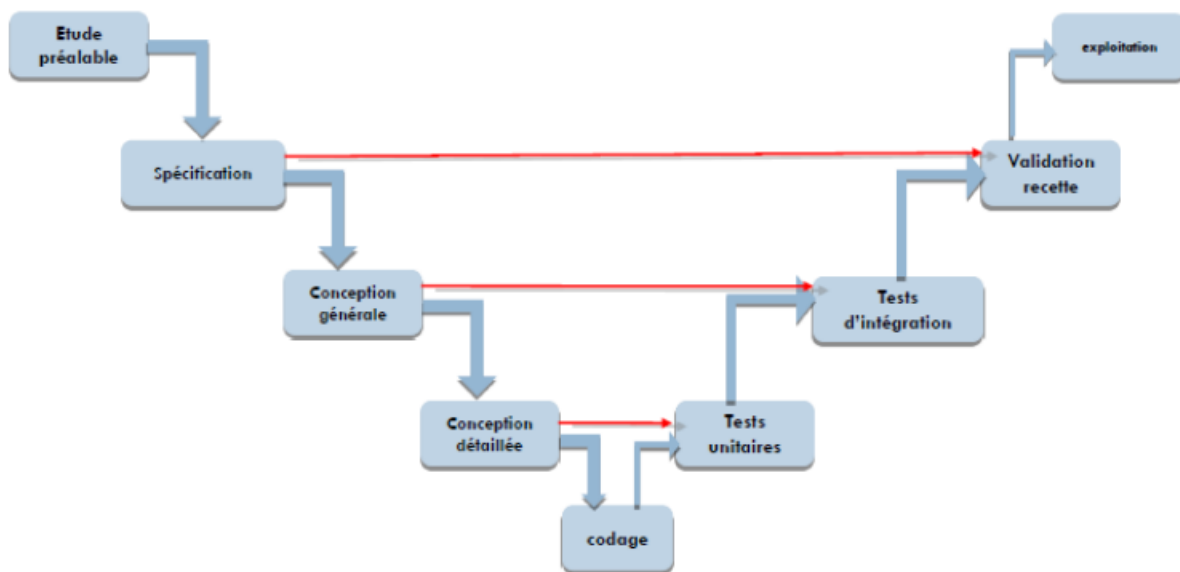
2. Spécification (Requirements) : Description précise des tâches à réaliser ainsi que des contraintes à respecter. Planifier les étapes à suivre ;
 3. Conception générale (Product Design) : Déterminer la structure et l'architecture du système, ainsi que la façon dont les différentes parties vont communiquer entre – elles. Pour cela, on procède par une décomposition en module ;
 4. Conception détaillée (Detailed Design) : Raffinement de la conception générale jusqu'à l'obtention d'une forme permettant l'écriture immédiate du programme ;
 5. Codage (Coding) : Ecriture du programme proprement dit (Code source) ;
 6. Intégration : Regroupement des différents modules et construction de l'architecture générale ;
 7. Validation globale/Recette : On valide par des tests successifs ;
 8. Diffusion : Préparation et diffusion des différentes versions ;
 9. Exploitation : Déploiement ou mise en place du système dans l'environnement opérationnel.
- Un constat qui saute aux yeux est qu'une fois le processus de développement est lancé, il est difficile, voire impossible de faire marche arrière étant donné que l'étape suivante est conditionnée par la fin de l'étape qui précède : Ce qui ne permet pas un développement adaptatif aux changements tout au long du développement. En outre, ce modèle de développement impose une importante réflexion sur les choix faits en amont car le coût de la correction d'une erreur est toujours important. C'est pourquoi, il est bien adapté pour des projets de petite taille et dont le domaine est bien maîtrisé.

II.2 MODELE EN V

La représentation en V tient d'avantage compte de la réalité, le processus de développement n'est pas réduit à un enchaînement de tâches séquentielles. Elle montre que :

1. C'est en phase de spécification que l'on se préoccupe des procédures de qualification ;
2. C'est en phase de conception globale que l'on se préoccupe des procédures d'intégration ;
3. C'est en phase de conception détaillée que l'on prépare les tests unitaires. Le modèle de cycle de vie en V permet d'anticiper sur les phases ultérieures de développement du produit. En particulier le modèle en V permet de commencer plus tôt, c'est – à – dire le plan de test de qualification et le plan d'évaluation des performances. Le modèle en V comme celui en cascade

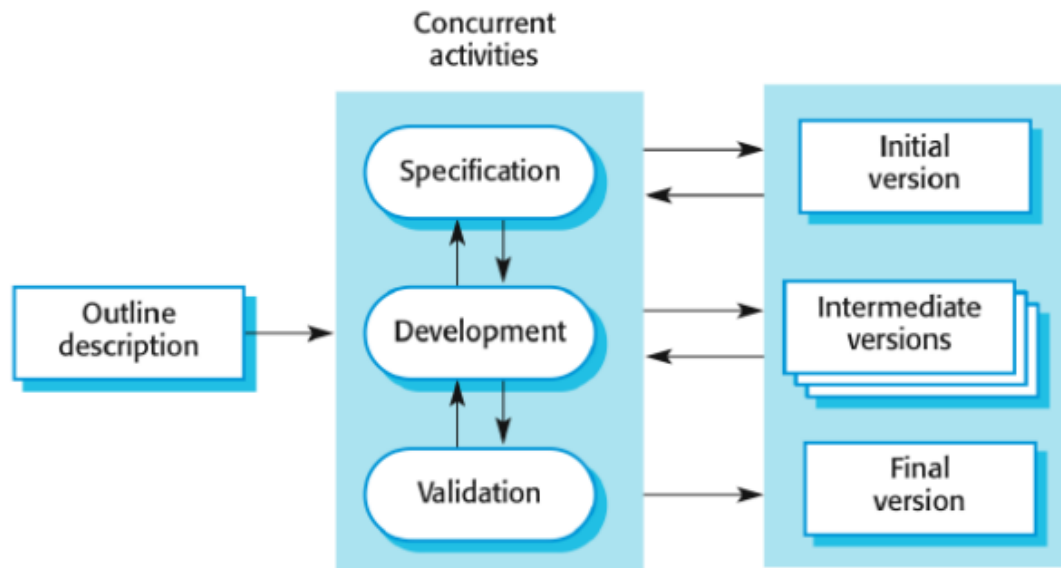
conduisent à commencer plus tôt la documentation utilisateur. Les deux modèles permettent de développer parallèlement différents modules lorsque la phase de conception globale est validée. En outre, beaucoup des choix techniques sont pris en compte sans disposer d'informations concrètes (Prise des risques) et il faut attendre longtemps pour savoir si on a bien construit le logiciel, car une fois que le codage a commencé, il est difficile de faire marche arrière. En plus, ce modèle n'est pas adapté aux systèmes non fonctionnels (Dont on ne connaît pas a priori tous les besoins de premier rang d'un utilisateur).



II.3 DEVELOPPEMENT INCREMENTAL

Ce modèle de cycle de vie prend en compte le fait qu'un logiciel peut être construit étape par étape. Le logiciel est spécifié et conçu dans son ensemble. La réalisation se fait par 15 incréments de fonctionnalités. Chaque incrément est intégré à l'ensemble des précédents et à chaque étape le produit est testé, exploité et maintenu dans son ensemble. Ce cycle de vie permet de prendre en compte l'analyse de risques et de faire accepter progressivement un logiciel par les utilisateurs plutôt que de faire un changement brutal des habitudes.

Exemple : Un scheduler (ordonnanceur) ou un gestionnaire de fichiers / Et ou gestionnaire de processus peut constituer des incréments d'un Système d'Exploitation.

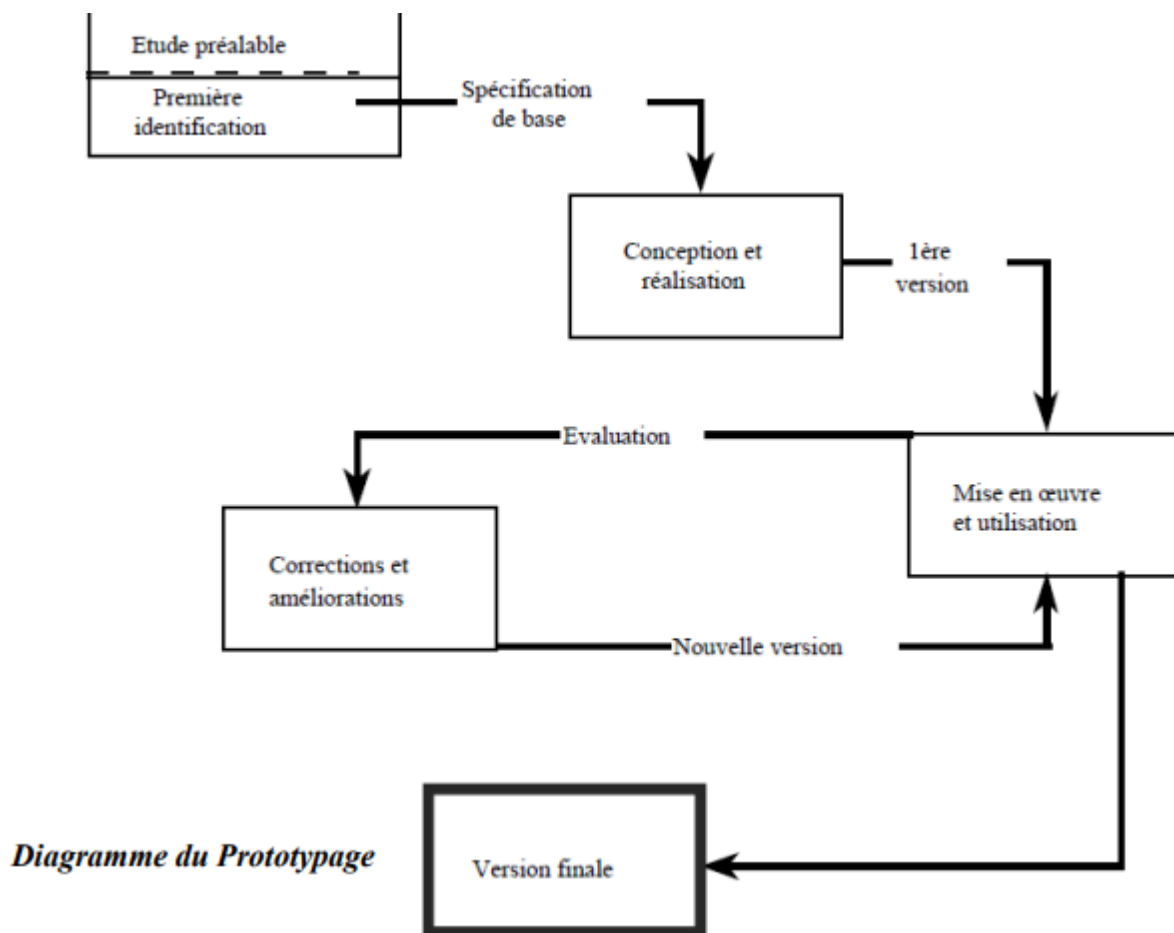


Ce modèle a en outre les caractéristiques suivantes :

- On hiérarchise les besoins du client en termes de priorité : Ce qui permet de satisfaire les besoins fonctionnels (Les plus importants) de l'utilisateur ;
- Chaque itération du modèle vise à obtenir un ensemble de fonctionnalités par ordre de priorité : ce qui permet d'avoir des feedbacks réguliers de l'utilisateur ;
- Traiter les parties les plus critiques du système en premier minimise les risques d'inadéquation avec le produit final : Ce qui permet de livrer plus rapidement des morceaux de logiciels utiles au client ;
- Cependant, il se peut que les choix pris en amont, trop focalisés sur ce noyau de fonctionnalités, compromettent le développement des fonctionnalités secondaires : Ce qui rallonge le temps d'exécution du projet ainsi que son coût pour reconstruire le logiciel et l'améliorer. Les changements réguliers conduisent à une détérioration de la structure du logiciel, c'est – à – dire que plus on incorpore des changements, plus il devient difficile et coûteux. Pour ce modèle, il est possible de développer les différents incréments en parallèle mais ceci peut être dangereux car on ne profite plus de l'aspect incrémental même si on accélère le développement. Si le nombre d'incrémentes n'est pas assez important ce modèle de cycle de vie perd de son intérêt et peut se rapprocher d'une approche par essai erreur à déconseiller.

II.4 PROTOTYPAGE

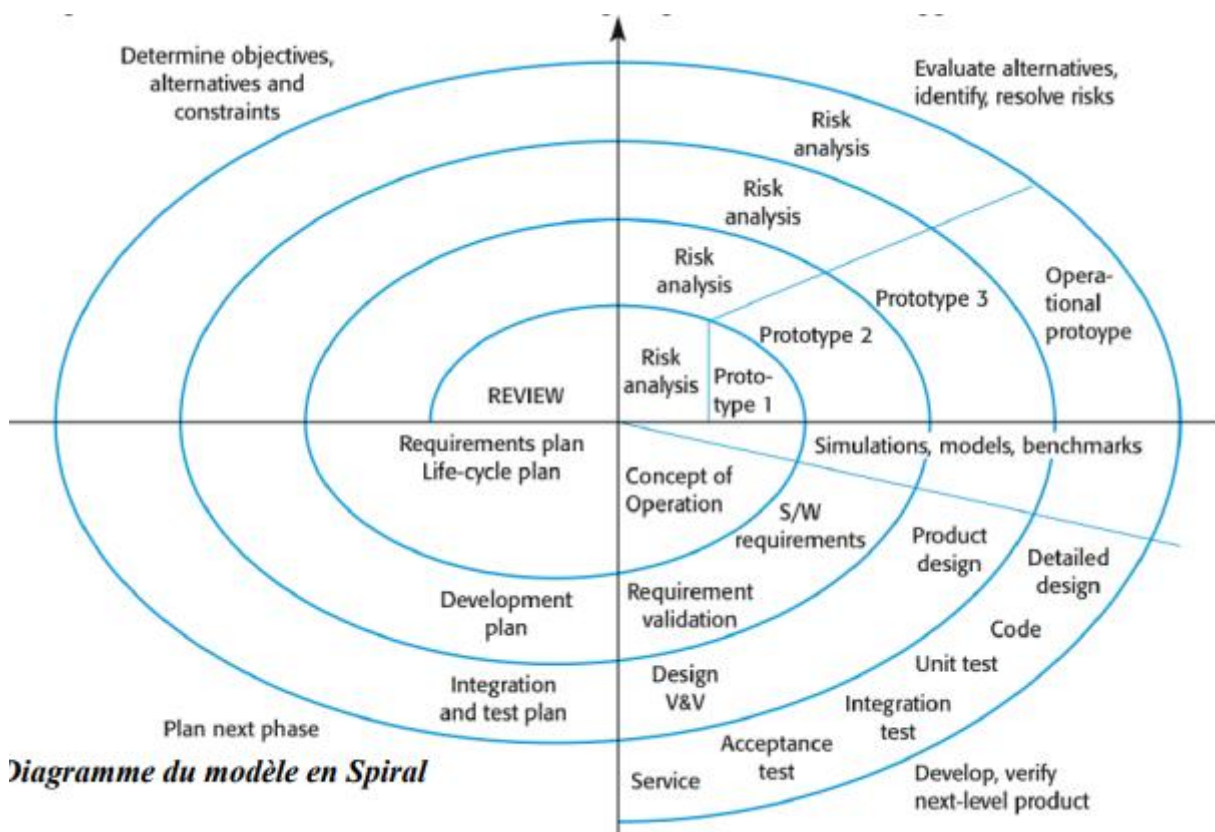
La maquette ou prototype permet au client et au développeur de bien se mettre d'accord sur la nature du produit à réaliser (Degré élevé de participation de l'utilisateur), et en particulier sur l'interface et les fonctionnalités. La notion de rapidité est importante car cette phase conditionne toute la suite du cycle de vie et permet de raccourcir la durée des allers retours entre client et développeur pendant la phase d'analyse des besoins. La première version du prototype est l'embryon (Un modèle exécutable d'un système logiciel, qui souligne des aspects spécifiques) du produit final, puis on itère jusqu'au produit final : Dans le schéma, cet embryon représente la première version du logiciel.



La plus grande différence entre le modèle incrémental et celui prototypé est que le premier est fait par bloc de fonctionnalité (Une fonctionnalité seulement peut à elle seule constitué un incrément), tandis que le second englobe un grand nombre des fonctionnalités sous forme d'un prototype (Qui à lui seul constitue déjà une version du logiciel). C'est ainsi que l'on peut avoir un développement prototypé et incrémental au même moment (Dans lequel chaque prototype est fait sous forme incrémental, donc chaque incrément est validé sous forme de version du logiciel).

II.5 MODELE EN SPIRAL

Proposé par B. Boehm en 1988, ce modèle de cycle de vie tient compte de la possibilité de réévaluer les risques en cours de développement, il emprunte au prototypage incrémental mais lui adjoint une dimension relevant de la prise de décision managériale et non purement technique. Il couvre l'ensemble du cycle de développement d'un produit. Il met l'accent sur l'activité d'analyse des risques. En somme, c'est un modèle Prototypo – Incrémental. Dans ce modèle, le processus de développement est représenté par une spirale plutôt qu'une séquence d'activités avec retour arrière éventuels ; tout en sachant que chaque boucle dans la spirale représente une étape du processus de développement. De cette façon, les risques sont explicitement adressés et résolus tout au long du processus de développement.



En outre, ce modèle peut s'expliquer par les étapes suivantes :

- Détermination des objectifs du cycle, des alternatives pour les atteindre et des contraintes ; à partir des résultats des précédents, ou de l'analyse préliminaire des besoins ;
- Analyse des risques, évaluation des alternatives à partir de maquettage et/ou prototypage ;

- Développement et vérification de la solution retenue, un modèle « classique » (Cascade ou en V) peut être utilisé ici ;
- Revue des résultats et vérification du cycle suivant.

II.6 MODELE ORIENTE REUTILISATION

Ce modèle est de plus en plus utilisé de nos jours, et est basée sur une réutilisation systématique de composants existants (Commercial-Off-The-Shelf – COTS) pour concevoir un nouveau système. La volonté de réutilisation du code induit des cycles de vie légèrement différents de ceux vus jusqu'ici. Les objets décrits dans un projet peuvent être réutilisés dans un autre et sont donc récoltés en fin de cycle de vie pour être placés dans une bibliothèque d'objets ou pour être plus précis bibliothèque des classes. Il est important de consacrer une part non négligeable du temps du projet à gérer la réutilisation : Ceci induit un gain de coût (D'argent) ou de ré – développement et facilite l'adaptation aux changements. Pour arriver à un pareil développement, il est important de suivre les étapes suivantes :

- Analyse des composants : On fait une analyse préalable des composants à utiliser ;
- Spécification des modifications : On spécifie ce qui devrait être modifié ;
- Conception avec réutilisation : Conception des différents modules de réutilisabilité ;
- Développement et intégration : Intégration proprement dite des différents modules de réutilisation dans le projet. Les types des composants logiciels les plus courants sont :
 - Les Web services : qui permettent de développer selon des standards et qui sont disponibles par des appels sur un serveur ;
 - Collections d'objets intégrés dans un Framework (tel que .NET ou JEE) : Un Framework est une bibliothèque personnalisée et respectant des règles strictes de développement ;
 - Logiciels autonomes (COTS) configurés pour une utilisation dans un environnement particulier.

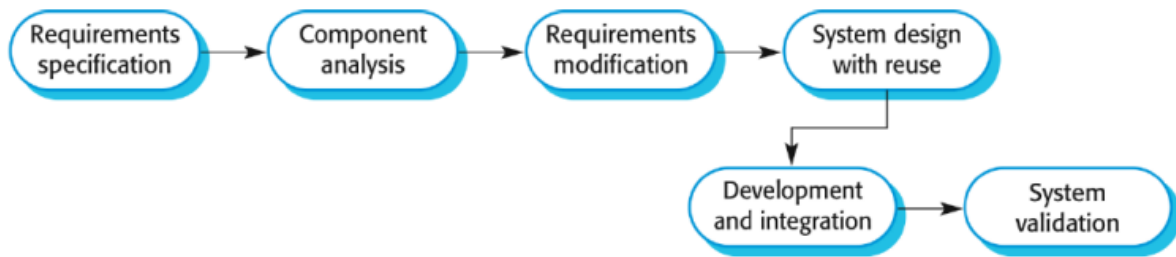


Figure6. Diagramme du modèle Orienté réutilisation

II.7. EN PRATIQUE

Dans la pratique, le développement d'un logiciel ne suit pas toujours de façon stricte un modèle quelconque ; il peut ainsi y avoir une association des différents modèles pour aboutir à un résultat. Tous ces modèles nous servent juste de guide pour une bonne planification quant au développement des logiciels.

CHAPITRE III : INTRODUCTION AUX PATRONS DE CONCEPTION

III.1 DEFINITION

Un Design Pattern ou Patron de Conception est tout d'abord un Modèle de Conception décrivant un certain nombre des principes et critères. Christopher Alexander dit que « Chaque modèle décrit un problème qui se produit très souvent dans notre environnement, puis décrit le cœur de la solution à ce problème, de telle sorte que vous pouvez utiliser cette solution un millier de fois, sans jamais le faire deux fois de la même manière ». Dans ce contexte, Alexander parlait des modèles dans les bâtiments et les villes, ce qui est aussi vrai à propos des modèles de conception Orienté Objet. Pour nous, les solutions sont exprimées en termes d'objets et d'interfaces au lieu de murs et de portes, mais au cœur des deux types des modèles et dans un contexte, c'est une solution à un problème. C'est ainsi que pour lui, un Design Pattern n'est rien d'autre que la description d'un problème fréquent en conception et la solution à ce problème de façon à être utilisé plusieurs fois sans l'adapter deux fois de la même manière. En synthèse, nous pouvons définir le Design Pattern comme étant une solution réutilisable à un problème très récurrent en termes de conception Orienté Objet.

III.2 POURQUOI LE DESIGN PATTERNS

L'utilisation des Designs Pattern est le résultat des plusieurs années d'expérience dans la réalisation des différents projets informatique basés sur la Programmation Orienté Objet. C'est ainsi que nous pouvons énumérer une liste (Non exhaustive) de leurs bien fondé.

- Ils sont des solutions standardisées à des problèmes de conception ;
- Ils sont des solutions indépendantes des langages de programmation ;
- Ils sont proposés par des experts en POO ;
- Ils sont utiles pour des applications de grande taille ;
- Ils sont orientés vers un bon découpage en package (Modularité et Réutilisabilité) ;
- Ils reposent beaucoup sur les interfaces.

III.3 CATEGORIE DES DESIGN PATTERNS

III.3.1 Design Patterns de création

Ils permettent de décrire la manière dont un objet ou un ensemble d'objets peuvent être créés, initialisés, et configurés. Ils permettent aussi une isolation du code relatif à la création, à l'initialisation afin de rendre l'application indépendante de ces aspects. Dans cette catégorie nous avons les cinq (5) Patterns suivants :

- Abstract Factory ;
- Factory Method;
- Builder;
- Prototype;
- Singleton.

III.3.2 Design Patterns de structure

Ils permettent de décrire la manière dont doivent être connectés des objets de l'application afin de rendre ces connections indépendantes des évolutions futures de l'application. Ils permettent en outre d'obtenir un découpage de l'interface et de l'implémentation de classes et d'objets. Dans cette catégorie nous avons les sept (7) Patterns suivants :

- Adapter ;
- Composite;
- Proxy;
- Bridge;
- Decorator;
- Facade;
- Flyweight.

III.3.3 Design Patterns de comportement

Ils permettent de décrire les comportements d'interaction entre objets ainsi que la gestion des interactions dynamiques entre des classes et des objets. Dans cette catégorie, nous avons les onze (11) Patterns suivants :

- Interpreter ;

- Template Method ;
- Chain of Responsibility ;
- Command;
- Iterator;
- Mediator;
- Memento;
- Observer;
- State ;
- Strategy ;
- Visitor.

III.3.4 Portée des Design Patterns

Suivant la portée, on distingue deux catégories des Patterns : Portée de Classe et Portée d'instance.

- Portée de Classe : Sont caractérisées par une focalisation sur les relations entre classes et leurs sous-classes. Elles permettent une bonne réutilisation par l'héritage ;
- Portée d'instance : Sont caractérisés par une focalisation sur les relations entre objets. Elles permettent une bonne réutilisation par composition. Voici un tableau récapitulatif donnant les types de Design Patterns avec leurs portées respectives.

III.4 APPLICATION D'UN DESIGN PATTERN

Pour pouvoir bien expliquer certains Patterns, nous allons chaque fois donner trois éléments : Problème (Problème que veut résoudre le Design Pattern), Solution (Solution envisagée par le Design Pattern) et Conséquences (Conséquence du Design Pattern).

III.4.1 Design Pattern Singleton

- Problème : Avoir une seule instance d'une classe et pouvoir y accéder et la manipuler facilement.

- Solution : Une seule classe est nécessaire pour écrire ce motif.

- Conséquence : L'unicité de l'instance est complètement contrôlée par la classe elle-même. Ce motif peut facilement être étendu pour permettre la création d'un nombre donné d'instances.

- Exemple de code : Nous donnons ci – bas le code correspondant à ce Pattern

```
using System.Linq;
using System.Text;

namespace TestPattern
{
    class Singleton
    {
        private static Singleton instance = null;

        //Constructeur privé pour la classe
        private Singleton()
        {
        }

        //Accesseur static sans Mutateur
        public static Singleton Instance
        {
            get
            {
                if (instance == null)
                    instance = new Singleton();
                return instance;
            }
        }

        //Methode static de la classe Singleton
        public static Singleton GetInstance()
        {
            if (instance == null)
                instance = new Singleton();
            return instance;
        }
    }
}
```

```

class Program
{
    static void Main(string[] args)
    {
        //Une seule instance de la classe en utilisant la methode
        //statique
        Singleton s = Singleton.GetInstance();
        //Même instance unique de la classe en utilisant le Getter
        Singleton s1 = Singleton.Instance;
    }
}

```

Nous cachons le constructeur par défaut de sorte que l'application entière peut seulement appeler la méthode d'instance statique publique pour accéder au singleton, ou utiliser la variable de propriété (Accesseur ou Getter) statique publique Instance. La variable statique et privée instance contient l'instance unique de la classe.

III.4.2 Design Pattern Composite

- Problème : Etablir des structures arborescentes entre des objets et les traiter uniformément.
- Conséquence :
 - Hiérarchies de classes dans lesquelles l'ajout de nouveaux composants est simple ;
 - Simplification du client qui n'a pas à se préoccuper de l'objet accédé ;
 - Mais, il est difficile de restreindre et de vérifier le type de composants.

III.4.3 Design Pattern Decorator

- Problème : Ajout dynamique de fonctionnalités à un objet.
- Solution :
 - Intégrer toutes les fonctionnalités dans la classe ;
 - Utiliser des conditionnelles pour utiliser une fonctionnalité ou pas.

III.4.4. Design Pattern Observer

- Problème :
 - Avoir un objet qui varie.

- Dépendance vers d'autres composants.
- Solution : Intégrer les éléments dépendant en dur dans la classe modifiable.

III.4.5 Design Pattern Strategy

- Problème :
 - On veut définir une famille d'algorithmes ;
 - Encapsuler chacun et les rendre interchangeables tout en s'assurant que chaque algorithme peut évoluer indépendamment des clients qui l'utilisent.
- Solution : Utiliser une classe qui représente la stratégie et le comportement associé.

III.4.6 Design Pattern Factory Method

- Problème :
 - Création d'un objet, mais difficile de connaître précisément de quelle classe.
 - Ce motif est à utiliser dans les situations où existe le besoin de standardiser le modèle architectural pour un ensemble d'applications, tout en permettant à des applications individuelles de définir elles-mêmes leurs propres objets à créer
- Solution : Définir une interface pour créer un objet, mais laissez les sous-classes décider de la classe à instancier. Factory Method permet à une classe de différer l'instanciation des sous-classes.